



# Fundament Backend

Универсальная платформа клиентов,  
расписания и задач

Подготовлено Codex  
Август 2026

Содержание: резюме · ключевые решения · последствия · рекомендации · приложение

|                        |   |
|------------------------|---|
| Краткое резюме .....   | 2 |
| Ключевые решения ..... | 3 |
| Последствия .....      | 3 |
| Рекомендации .....     | 5 |
| Заключение .....       | 5 |
| Приложение .....       | 6 |

# Краткое резюме

Спроектирован и реализован универсальный мультитенантный backend на Django REST Framework для компаний, работающих с клиентами, сотрудниками, задачами и расписанием. Каждая рабочая сущность изолирована по компании; роли, статусы, поля и настройки меняются без переписывания ядра.

Результат включает Django Admin, REST API, интерактивную OpenAPI-документацию, PostgreSQL-конфигурацию, поиск и фильтры, аудит изменений, перенос и отмену записей, контроль пересечений сотрудников и ресурсов, миграции и автоматические тесты.

## Коротко

- Покрытие: компании, пользователи, членства, роли, клиенты, услуги, ресурсы, графики, отсутствие, записи, задачи, комментарии, статусы и аудит.
- Расширяемость: JSON-поля для индивидуальных данных, произвольные роли/статусы, feature flags и branding в настройках компании.
- Интеграция: версионированный API /api/v1/, Swagger /api/docs/ и обязательный tenant-контекст X-Company-ID.

## Исходные требования

Спецификация Notion требует общий фундамент для разных отраслей: управление клиентами и сотрудниками, права, рабочие дни, расписание, записи, задачи, поиск, мониторинг, история и настройки компании. Фронтенд отложен; backend должен быть готов к последующему подключению.

Пример интерфейса клиники дополнительно показал услуги, длительность и цену, доступность сотрудников, кабинеты/оборудование и риск двойного бронирования. Эти элементы реализованы как необязательные универсальные сущности Service и Resource.

# Ключевые решения

Архитектура ориентирована на повторное использование одной кодовой базы разными заказчиками при строгой изоляции данных и настройке поведения через данные.

## Контекст и ограничения

В первой версии используется стандартный пользователь Django и отдельное членство Membership. Это сохраняет совместимость с Django Admin и позволяет одному аккаунту иметь разные роли в разных компаниях. SQLite доступен для разработки, PostgreSQL включается переменными окружения.

## Закономерности в требованиях

Клиенты, записи и задачи различаются между отраслями в основном названиями, дополнительными полями, статусами и правами. Поэтому стабильные связи хранятся в нормализованных моделях, а изменяемые атрибуты — в `extra_data/settings`. Такой баланс сохраняет целостность и гибкость.

Главный вывод. Компания является границей безопасности и конфигурации; каждый API-запрос к рабочим данным проверяет членство и активную компанию.

## Последствия

Frontend сначала получает доступные компании, затем передаёт X-Company-ID. Доступ к действию определяется строковыми разрешениями роли, например `appointments.manage`; роль с \* получает все API-права. Django Admin остаётся служебной панелью, а продуктовые панели могут строиться поверх того же API.

| Theme              | Observation          | Implication                                           |
|--------------------|----------------------|-------------------------------------------------------|
| Мультиотнашенность | Company + Membership | Изоляция заказчиков и разные роли одного пользователя |

| Theme      | Observation                 | Implication                                |
|------------|-----------------------------|--------------------------------------------|
| Гибкость   | Роли, статусы и JSON-поля   | Адаптация без форка ядра и лишних миграций |
| Расписание | Сотрудники, услуги, ресурсы | Контроль занятости и двойного бронирования |

# Рекомендации

Код готов как фундамент, но перед публичным production-релизом нужны инфраструктурные решения конкретного заказчика: домен, PostgreSQL, HTTPS, резервные копии, мониторинг и политика хранения персональных данных.

1. Запустить базу. Создать .env, применить миграции и выполнить bootstrap\_company; затем настроить роли, статусы, услуги и графики через админку.
2. Подключить frontend. Использовать OpenAPI-схему, добавить выбранный механизм токенов/SSO, передавать X-Company-ID и строить элементы интерфейса по доступным разрешениям.
3. Усилить выпуск. Добавить CI, PostgreSQL-интеграционные тесты, rate limiting, централизованные логи, Sentry/аналог, фоновые уведомления и резервное восстановление.

## Заключение

Фундамент покрывает ядро ежедневной работы и не фиксирует отраслевые названия в коде. Клинический пример является одной конфигурацией общей модели, а не отдельной архитектурой.

На следующем этапе целесообразно согласовать контракт авторизации и конкретные экраны frontend, после чего API можно расширять предметными модулями без изменения базовой tenant-модели.

# Приложение

## Карта реализации

Модели находятся в `app/models.py`; API и бизнес-действия — `app/views.py`; сериализация и валидация — `app/serializers.py`; роли — `app/permissions.py`; административная панель — `app/admin.py`.

Проверки: миграции 0001 и 0002 применены; `django check` проходит; четыре автоматических теста проверяют `tenant`-изоляцию, запрет роли, пересечение записей и чужую компанию; OpenAPI-схема валидируется без ошибок.

## Источники

- Notion: «Фундамент универсальной системы для работы с клиентами и задачами», получено 12 августа 2026.
- `scheduling-platform (1).html`: пример интерфейса расписания клиники, локальный файл заказчика.
- `README.md` и `schema.yml`: эксплуатационная инструкция и контракт реализованного API.
- Django Admin: встроенная служебная панель управления пользователями и всеми рабочими сущностями.

Версия отчёта: 1.0 · 16 августа 2026